```python
# Practical 2: Game Environment for Cognitive System

# Define grid size
ROWS = 5
COLS = 5

# Define environment
grid = [
    ['A', '0', '0', '0', '0'],
    ['0', 'X', 'X', '0', '0'],
    ['0', '0', '0', 'X', '0'],
    ['X', '0', '0', '0', '0'],
    ['0', '0', 'X', '0', 'G']
]

# Find agent position
def find_agent(grid):
    for i in range(ROWS):
        for j in range(COLS):
            if grid[i][j] == 'A':
                return i, j

# Display environment
def display_grid():
    for row in grid:
        print(" ".join(row))
    print()

# Move agent
def move_agent(direction):
    x, y = find_agent(grid)
    new_x, new_y = x, y

    if direction == 'up':
        new_x -= 1
    elif direction == 'down':
        new_x += 1
    elif direction == 'left':
        new_y -= 1
    elif direction == 'right':
        new_y += 1

    # Check valid move
    if 0 <= new_x < ROWS and 0 <= new_y < COLS and grid[new_x][new_y] != 'X':
        if grid[new_x][new_y] == 'G':
            print("🎉 Goal Reached!")
            exit()
        grid[x][y] = '0'
        grid[new_x][new_y] = 'A'

# Main loop
display_grid()
while True:
    move = input("Enter move (up/down/left/right): ").lower()
    move_agent(move)
    display_grid()
```

```python
# Practical 3: Implementing Simple Trees for a Cognitive System

# Define Node class
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

# Create tree structure
def create_tree():
    root = Node("Is it raining?")

    root.left = Node("Take Umbrella")
    root.right = Node("No Umbrella Needed")

    return root

# Traverse tree (Cognitive Decision)
def traverse_tree(node):
    if node.left is None and node.right is None:
        print("Decision:", node.value)
        return

    answer = input(node.value + " (yes/no): ").lower()

    if answer == "yes":
        traverse_tree(node.left)
    else:
        traverse_tree(node.right)

# Main Program
tree = create_tree()
traverse_tree(tree)
```

```python
# Practical 4: Text Analytics using POS Tagger (Simple Implementation)

import random
import numpy as np

# Sample POS-tagged corpus (word, POS)
corpus = [
    [("I", "PRP"), ("love", "VBP"), ("machine", "NN"), ("learning", "NN"), ("techniques", "NNS")],
    [("Text", "NN"), ("analytics", "NNS"), ("is", "VBZ"), ("very", "RB"), ("useful", "JJ")]
]

WINDOW_SIZE = 5
EMBEDDING_SIZE = 4

# Step 1: Create vocabulary
vocab = set()
for sentence in corpus:
    for word, tag in sentence:
        vocab.add(word)

vocab = list(vocab)
V = len(vocab)

# Step 2: Word to index mapping
word_to_index = {word: idx for idx, word in enumerate(vocab)}

# Step 3: Initialize embedding matrix
E = np.random.rand(V, EMBEDDING_SIZE)

# Step 4: Generate training windows
def generate_windows(sentence):
    words = [word for word, tag in sentence]
    tags = [tag for word, tag in sentence]

    windows = []
    for i in range(2, len(words) - 2):
        window = words[i-2:i+3]
        label = tags[i]
        windows.append((window, label))
    return windows

# Step 5: Display windows and POS tags
print("Training Samples:\n")
for sentence in corpus:
    samples = generate_windows(sentence)
    for window, tag in samples:
        print("Window:", window, "=> POS Tag:", tag)

# Step 6: Display embedding for a word
sample_word = vocab[0]
print("\nWord Embedding for:", sample_word)
print(E[word_to_index[sample_word]])
```

```python
# Practical 5: Image Analytics without OpenCV

from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Load Image
image = Image.open("kastura.jpeg")

# Step 2: Convert to Grayscale
gray_image = image.convert("L")

# Convert to numpy array
img_array = np.array(gray_image)

# Step 3: Simple Edge Detection using Gradient
# Calculate difference between adjacent pixels
edges = np.abs(np.diff(img_array, axis=0))

# Step 4: Display Images
plt.figure(figsize=(10,5))

plt.subplot(1,3,1)
plt.title("Original")
plt.imshow(image)
plt.axis("off")

plt.subplot(1,3,2)
plt.title("Grayscale")
plt.imshow(gray_image, cmap="gray")
plt.axis("off")

plt.subplot(1,3,3)
plt.title("Edges")
plt.imshow(edges, cmap="gray")
plt.axis("off")

plt.show()

# Step 5: Histogram Analysis
plt.hist(img_array.ravel(), bins=256)
plt.title("Image Intensity Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.show()
```

```python
# Practical 6: Hidden Markov Model Implementation

import numpy as np

states = ("Rainy", "Sunny")
observations = ("Walk", "Shop", "Clean")

# Initial Probabilities
start_probability = {"Rainy": 0.6, "Sunny": 0.4}

# Transition Probabilities
transition_probability = {
    "Rainy": {"Rainy": 0.7, "Sunny": 0.3},
    "Sunny": {"Rainy": 0.4, "Sunny": 0.6},
}

# Emission Probabilities
emission_probability = {
    "Rainy": {"Walk": 0.1, "Shop": 0.4, "Clean": 0.5},
    "Sunny": {"Walk": 0.6, "Shop": 0.3, "Clean": 0.1},
}

def viterbi(obs, states, start_p, trans_p, emit_p):
    V = [{}]

    # Initialize base cases
    for st in states:
        V[0][st] = start_p[st] * emit_p[st][obs[0]]

    # Run Viterbi for t > 0
    for t in range(1, len(obs)):
        V.append({})
        for st in states:
            max_tr_prob = max(V[t-1][prev_st] * trans_p[prev_st][st] for prev_st in states)
            V[t][st] = max_tr_prob * emit_p[st][obs[t]]

    return V

# Example observation sequence
obs_sequence = ["Walk", "Shop", "Clean"]

result = viterbi(obs_sequence, states, start_probability, transition_probability, emission_probability)

print("Viterbi Probability Table:")
for step in result:
    print(step)
```

```python
# Practical 7: Viterbi Algorithm Implementation

states = ["Rainy", "Sunny"]
observations = ["Walk", "Shop", "Clean"]

# Initial Probabilities (π)
start_prob = {"Rainy": 0.6, "Sunny": 0.4}

# Transition Probabilities (A)
trans_prob = {
    "Rainy": {"Rainy": 0.7, "Sunny": 0.3},
    "Sunny": {"Rainy": 0.4, "Sunny": 0.6}
}

# Emission Probabilities (B)
emit_prob = {
    "Rainy": {"Walk": 0.1, "Shop": 0.4, "Clean": 0.5},
    "Sunny": {"Walk": 0.6, "Shop": 0.3, "Clean": 0.1}
}

def viterbi(obs, states, start_p, trans_p, emit_p):
    V = [{}]          # Probability table
    backpointer = {}  # To store best previous state

    # Initialization
    for state in states:
        V[0][state] = start_p[state] * emit_p[state][obs[0]]
        backpointer[state] = [state]

    # Recursion
    for t in range(1, len(obs)):
        V.append({})
        new_path = {}

        for curr_state in states:
            (max_prob, prev_state) = max(
                (V[t-1][state] * trans_p[state][curr_state] * emit_p[curr_state][obs[t]], state)
                for state in states
            )

            V[t][curr_state] = max_prob
            new_path[curr_state] = backpointer[prev_state] + [curr_state]

        backpointer = new_path

    # Termination
    (final_prob, final_state) = max(
        (V[len(obs)-1][state], state) for state in states
    )

    return backpointer[final_state], final_prob

# Observation sequence
obs_seq = ["Walk", "Shop", "Clean"]

best_path, probability = viterbi(obs_seq, states, start_prob, trans_prob, emit_prob)

print("Most Probable State Sequence:", best_path)
print("Probability:", probability)
```

```python
# Practical 8

import numpy as np

# Sample tagged corpus
corpus = [
    [("I", "PRP"), ("love", "VBP"), ("NLP", "NN"), ("very", "RB"), ("much", "RB")]
]

WINDOW_SIZE = 5
EMBEDDING_SIZE = 4
HIDDEN_SIZE = 8

# Step 1: Create vocabulary
vocab = set()
pos_tags = set()

for sentence in corpus:
    for word, tag in sentence:
        vocab.add(word)
        pos_tags.add(tag)

vocab = list(vocab)
pos_tags = list(pos_tags)

V = len(vocab)
T = len(pos_tags)

word_to_index = {word: i for i, word in enumerate(vocab)}
tag_to_index = {tag: i for i, tag in enumerate(pos_tags)}

# Step 2: Initialize embedding matrix (V x d)
E = np.random.rand(V, EMBEDDING_SIZE)

# Step 3: Initialize neural network weights
W1 = np.random.rand(WINDOW_SIZE * EMBEDDING_SIZE, HIDDEN_SIZE)
W2 = np.random.rand(HIDDEN_SIZE, T)

def softmax(x):
    exp_x = np.exp(x - np.max(x))
    return exp_x / exp_x.sum()

def create_window(sentence):
    words = [w for w, t in sentence]
    tags = [t for w, t in sentence]

    windows = []
    for i in range(2, len(words) - 2):
        window_words = words[i-2:i+3]
        target_tag = tags[i]
        windows.append((window_words, target_tag))
    return windows

# Generate training samples
samples = create_window(corpus[0])

for window, tag in samples:
    # Create input vector
    embeddings = [E[word_to_index[word]] for word in window]
    x = np.concatenate(embeddings)

    # Forward Pass
    z1 = np.dot(x, W1)
    a1 = np.tanh(z1)
    z2 = np.dot(a1, W2)
    output = softmax(z2)

    print("Window:", window)
    print("Predicted POS Probabilities:", output)
    print("Actual POS:", tag)
    print()
```

```python
# Practical 9

import numpy as np
import random

# Define environment
states = 9   # 3x3 grid
actions = 4  # up, down, left, right

Q = np.zeros((states, actions))

learning_rate = 0.8
discount_factor = 0.95
epsilon = 0.2
episodes = 500

# Reward matrix
rewards = np.full((states, actions), -1)
goal_state = 8

# Define transitions manually
def get_next_state(state, action):
    row = state // 3
    col = state % 3

    if action == 0 and row > 0: row -= 1      # up
    if action == 1 and row < 2: row += 1      # down
    if action == 2 and col > 0: col -= 1      # left
    if action == 3 and col < 2: col += 1      # right

    return row * 3 + col

# Training loop
for episode in range(episodes):
    state = random.randint(0, 8)

    while state != goal_state:
        if random.uniform(0,1) < epsilon:
            action = random.randint(0,3)
        else:
            action = np.argmax(Q[state])

        next_state = get_next_state(state, action)

        reward = 100 if next_state == goal_state else -1

        Q[state, action] = Q[state, action] + learning_rate * (
            reward + discount_factor * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state

print("Learned Q-Table:")
print(Q)
```

```python
# Practical 10: Cognitive Healthcare Symptom Analyzer

# Knowledge Base (Corpus)
disease_db = {
    "Flu": ["fever", "cough", "headache"],
    "Cold": ["sneezing", "cough"],
    "Migraine": ["headache", "nausea"],
    "COVID-19": ["fever", "cough", "loss of smell"]
}

def analyze_symptoms(user_symptoms):
    results = {}

    for disease, symptoms in disease_db.items():
        match_count = len(set(user_symptoms) & set(symptoms))
        confidence = match_count / len(symptoms)
        if match_count > 0:
            results[disease] = round(confidence * 100, 2)

    return results

# User Input
input_symptoms = input("Enter symptoms separated by comma: ").lower().split(",")

# Clean input
input_symptoms = [sym.strip() for sym in input_symptoms]

# Analyze
predictions = analyze_symptoms(input_symptoms)

# Output
if predictions:
    print("\nPossible Conditions:")
    for disease, confidence in predictions.items():
        print(f"{disease} - Confidence: {confidence}%")
else:
    print("No matching disease found. Please consult a doctor.")
```